



SERVICIO NACIONAL DE APRENDIZAJE - SENA

Análisis y Desarrollo de Software

VERIFICACIÓN DE PROCEDIMIENTOS PARA LA DEFINICIÓN DE COMPONENTES FRONT-END

Evidencia GA7-220501096-AA4-EV02

Proyecto de referencia

SAAAF - Sistema de Asignación de Actas de Activos Fijos. La evidencia define y justifica los componentes React que estructuran el front-end de los módulos trabajados en el proyecto.

Aprendiz	Brayan Giraldo García
Ficha	3235876
Programa	Análisis y Desarrollo de Software
Proyecto formativo	Construcción de software integrador de tecnologías orientadas a servicios
Fecha	18 de septiembre de 2026

Introducción

El diseño del front-end de una aplicación requiere definir componentes que representen de forma clara las necesidades funcionales, la navegación y las interacciones de las personas usuarias. En React, la interfaz se divide en componentes reutilizables y con responsabilidades específicas, lo que facilita la mantenibilidad, las pruebas y la evolución del software.

En el proyecto SAAAF, orientado a la administración de activos fijos y sus actas de asignación y devolución, el front-end debe soportar módulos de Dashboard, Inventario, Asignaciones, Empleados, Áreas y Actas. Esta evidencia documenta el procedimiento usado para definir los componentes React de dichas pantallas, explica la responsabilidad de cada componente y justifica su selección con base en las necesidades del sistema.

Objetivos

Objetivo general

Definir y justificar los componentes React necesarios para el front-end del sistema SAAAF, verificando que cada componente responda a las características, módulos y flujos funcionales del proyecto.

Objetivos específicos

- Identificar las necesidades de interfaz presentes en los módulos principales de SAAAF.
- Establecer componentes estructurales, reutilizables y de dominio con responsabilidades claras.
- Relacionar cada componente con datos, eventos, validaciones y módulos donde será utilizado.
- Justificar la selección de los componentes de acuerdo con criterios de reutilización, mantenibilidad, experiencia de usuario y coherencia funcional.
- Verificar mediante una matriz de trazabilidad que la propuesta cubre los criterios de evaluación de la evidencia.

1. Contexto del front-end de SAAAF

SAAAF es un sistema para apoyar el control de activos fijos mediante procesos de inventario, asociación de activos a empleados y áreas, generación de actas, consulta del estado de las asignaciones y registro de devoluciones. El front-end trabajado para el proyecto se organiza con React dentro de una aplicación Next.js, permitiendo construir pantallas modulares y una navegación coherente.

Módulo	Necesidad de interfaz
Dashboard	Resumen visual del sistema y acceso rápido a los módulos.
Inventario	Consulta, registro y actualización de activos.
Asignaciones	Selección de activo, empleado y área para formalizar una entrega.
Empleados	Consulta y mantenimiento de las personas que pueden recibir activos.
Áreas	Catálogo de áreas utilizadas para clasificar responsables y ubicaciones.
Actas	Consulta del documento de asignación, estado y proceso de devolución.

Criterio de diseño

La propuesta evita crear una pantalla monolítica. Los elementos repetitivos se convierten en componentes reutilizables y los procesos propios del negocio se encapsulan en componentes de dominio.

2. Procedimiento para definir los componentes front-end

Para verificar que la selección de componentes no sea arbitraria, se aplica el siguiente procedimiento antes de implementar una nueva pantalla o funcionalidad:

Paso	Verificación
1. Identificar el requisito	Se revisa la historia de usuario o necesidad del módulo y se determina qué debe poder ver o hacer la persona usuaria.
2. Separar responsabilidades	Se divide la pantalla en bloques con una sola responsabilidad: navegación, consulta, formulario, estado, confirmación o notificación.
3. Detectar reutilización	Se identifican elementos repetidos, como tablas, campos, botones, modales, filtros y etiquetas de estado.
4. Definir entradas y salidas	Se establecen props, datos requeridos, eventos y callbacks que conectarán cada componente con su componente padre.
5. Definir estado	Se determina qué información cambia durante la interacción: filtros, datos de formularios, carga, errores o selección actual.
6. Incorporar validaciones	Los formularios deben validar campos obligatorios, tipos de datos y reglas antes de enviar información.
7. Revisar experiencia y accesibilidad	Se comprueba navegación clara, respuesta visual, uso por teclado y adaptación a diferentes tamaños de pantalla.
8. Verificar trazabilidad	Cada componente se relaciona con una necesidad del proyecto y se justifica por su utilidad concreta.

3. Arquitectura propuesta de componentes React

La arquitectura se organiza en tres niveles: componentes estructurales, componentes reutilizables de interfaz y componentes de dominio. Esto permite que las páginas ensamblen piezas pequeñas en lugar de repetir código.

```
SAAAFApp
|-- AppShell
|   |-- SidebarNavigation
|   |-- TopHeader
|   |-- PageHeader
|-- DashboardPage -> SummaryCard + DataTable
|-- InventarioPage -> FilterBar + DataTable + AssetForm + StatusBadge
|-- EmpleadosPage -> DataTable + EntityForm
|-- AreasPage -> DataTable + EntityForm
|-- AsignacionesPage -> AssignmentForm + ConfirmDialog + ToastNotification
|-- ActasPage -> ActasTable + ActaDetail + ReturnForm
```

3.1 Componentes estructurales

Componente	Función en SAAAF	Datos / eventos	Justificación
AppShell	Contenedor general de la aplicación. Ubica navegación lateral, encabezado y contenido de la página.	children, estado de menú móvil.	Mantiene una estructura visual uniforme en todos los módulos y evita repetir la distribución base.
SidebarNavigation	Menú lateral con accesos a Dashboard, Inventario, Asignaciones, Empleados, Áreas y Actas.	ruta activa, ítems de navegación, evento de cierre en móvil.	Centraliza la navegación del sistema y facilita identificar el módulo activo.
TopHeader	Encabezado superior para identidad del sistema, usuario y acciones globales.	título, usuario, acciones.	Separa acciones globales de las acciones propias de cada módulo y mejora la orientación.
PageHeader	Título de la pantalla, descripción breve y acción principal.	title, description, action.	Unifica el inicio de cada módulo y permite presentar de forma consistente botones como Nuevo activo o Nueva asignación.

3.2 Componentes reutilizables de interfaz

Componente	Función	Props / eventos	Justificación
SummaryCard	Muestra un indicador del Dashboard, por ejemplo total de activos o actas.	label, value, icon, onClick.	Permite reutilizar el mismo patrón para diferentes indicadores sin duplicar estructura.
FilterBar	Agrupar búsqueda y filtros antes de una lista.	search, filters, onChange, onReset.	Inventario y Actas requieren localizar información rápidamente; un único componente mantiene el mismo comportamiento.
DataTable	Presenta colecciones en filas y columnas con acciones.	columns, rows, loading, onEdit, onView.	Activos, empleados, áreas y actas comparten el patrón de consulta tabular, por lo que conviene centralizarlo.
Pagination	Controla el desplazamiento entre páginas de resultados.	page, totalPages, onPageChange.	Evita cargar listas extensas en una sola vista y mantiene una experiencia estable.
FormField	Representa etiqueta, control, ayuda y error de validación.	label, name, value, error, onChange.	Reduce duplicación en formularios y estandariza la forma de mostrar validaciones.
StatusBadge	Muestra un estado con texto visualmente identificable.	status.	Facilita reconocer estados como disponible, asignado, devuelto o pendiente sin leer datos adicionales.
ModalDialog	Ventana para formularios o información que requiere foco temporal.	open, title, children, onClose.	Permite editar o revisar información sin perder el contexto de la lista actual.
ConfirmDialog	Solicita confirmación antes de una acción sensible.	open, message, onConfirm, onCancel.	Reduce errores en acciones como devolver, eliminar o cambiar un estado importante.
ToastNotification	Mensaje temporal de éxito, advertencia o error.	type, message.	Da retroalimentación inmediata después de guardar, actualizar o fallar una operación.
LoadingEmptyState	Representa carga, ausencia de resultados o error controlado.	loading, empty, error.	Evita pantallas ambiguas y comunica claramente qué está ocurriendo mientras se consultan datos.

3.3 Componentes de dominio de SAAAF

Componente	Responsabilidad	Módulo	Justificación
AssetForm	Registra o edita un activo: identificación, características y estado.	Inventario	Encapsula reglas y validaciones propias de los activos; evita dispersar esa lógica por la pantalla.
AssetsTable	Configura la tabla de inventario y sus acciones.	Inventario	Especializa DataTable con columnas y acciones del dominio, manteniendo reusable la tabla genérica.
EmployeeForm	Registra o modifica información del empleado receptor.	Empleados	Concentra validaciones de los datos del empleado y facilita reutilizar el formulario en altas y edición.
AreaForm	Gestiona el catálogo de áreas.	Áreas	Mantiene independiente el mantenimiento de áreas y permite usarlas después como opciones en asignaciones.
AssignmentForm	Selecciona activo, empleado y área, valida disponibilidad y crea la asignación.	Asignaciones	Es el núcleo del flujo de entrega; requiere estado propio y validaciones coordinadas entre varias entidades.
ActasTable	Lista actas y permite consultar su estado y detalle.	Actas	Separa la consulta de actas del detalle y facilita agregar filtros o paginación sin afectar otras pantallas.
ActaDetail	Presenta la información completa de una acta y sus acciones disponibles.	Actas	Agrupar datos relacionados en una vista consistente y evita mezclar detalle con la lógica del listado.
ReturnForm	Registra la devolución del activo y observaciones asociadas.	Actas / Devolución	Aísla el proceso de devolución, permitiendo validar el estado previo y actualizar correctamente la trazabilidad.

4. Selección coherente de componentes según las necesidades

La selección anterior se verifica cruzando necesidades funcionales con componentes concretos. De esta manera, cada componente existe porque resuelve una necesidad identificada del software y no únicamente por conveniencia visual.

Necesidad	Componentes	Módulo	Razón de selección
Consultar resumen del sistema	SummaryCard, LoadingEmptyState	Dashboard	Los indicadores requieren bloques reutilizables que puedan cargarse de forma independiente.
Buscar y filtrar activos	FilterBar, AssetsTable, Pagination	Inventario	Permite encontrar registros sin sobrecargar la pantalla y mantener una lista escalable.
Crear o editar un activo	AssetForm, FormField, ModalDialog, ToastNotification	Inventario	El formulario concentra captura y validación; modal y notificación mejoran la interacción.
Mantener empleados y áreas	DataTable, EmployeeForm, AreaForm, ConfirmDialog	Empleados / Áreas	Se reutiliza la misma base de listado y edición para catálogos con comportamiento similar.
Asignar un activo	AssignmentForm, FormField, ConfirmDialog, ToastNotification	Asignaciones	El flujo requiere selección de varias entidades, validación y confirmación antes de persistir.

Necesidad	Componentes	Módulo	Razón de selección
Consultar actas	FilterBar, ActasTable, ActaDetail, StatusBadge	Actas	La persona necesita localizar un acta, reconocer su estado y abrir el detalle sin perder el listado.
Registrar devolución	ReturnForm, ConfirmDialog, ToastNotification	Actas / Devolución	La devolución cambia el estado del activo y requiere una acción explícita, validada y confirmada.

5. Datos, estado y eventos principales

Además de la estructura visual, cada componente debe definir cómo recibe información y cómo comunica las acciones de la persona usuaria. En SAAAF se prioriza el flujo descendente de datos mediante props y el uso de callbacks para enviar acciones al componente padre o a la capa responsable de consumir la API.

Componente	Estado local probable	Entradas (props)	Eventos / salidas
FilterBar	texto de búsqueda y filtros	search, filters	onChange, onReset
DataTable	registros y configuración de columnas	rows, columns, loading	onView, onEdit, onDelete
AssetForm	datos del activo y errores	initialValues, errors	onChange, onSubmit, onCancel
AssignmentForm	activo, empleado, área y observaciones	assets, employees, areas	onSelect, onSubmit
ConfirmDialog	mensaje y estado de apertura	open, message	onConfirm, onCancel
ReturnForm	motivo, observaciones y fecha	acta, initialValues	onChange, onSubmit
ToastNotification	tipo y mensaje	type, message	onClose

6. Ejemplo simplificado de componente

El siguiente ejemplo ilustra cómo un componente de presentación puede recibir datos por props y emitir una acción sin contener lógica de negocio compleja:

```
function StatusBadge({ status }) {
  const labels = {
    disponible: 'Disponible',
    asignado: 'Asignado',
    devuelto: 'Devuelto'
  };

  return (
    <span className={`status-badge status-${status}`}>
      {labels[status] ?? status}
    </span>
  );
}
```

StatusBadge cumple una única responsabilidad: representar visualmente un estado. No consulta la base de datos ni modifica registros. Esta separación facilita reutilizarlo en Inventario, Asignaciones y Actas.

7. Reglas de implementación para los componentes

- Mantener una responsabilidad principal por componente y dividir los componentes que acumulen demasiada lógica.
- Usar nombres descriptivos y consistentes, por ejemplo AssetForm, AssignmentForm y ActaDetail.
- Reutilizar componentes genéricos antes de duplicar tablas, campos, botones, modales o mensajes.

- Mantener la lógica de acceso a datos separada de los componentes puramente visuales siempre que sea posible.
- Validar los formularios antes de enviar datos y mostrar errores junto al campo correspondiente.
- Incluir estados de carga, error y ausencia de resultados en las consultas.
- Diseñar las pantallas para escritorio y tamaños reducidos, haciendo colapsable la navegación cuando sea necesario.
- Mantener accesibilidad básica mediante etiquetas, foco visible, controles de teclado y mensajes comprensibles.
- Probar los eventos principales: crear, editar, buscar, filtrar, asignar, consultar, acta y devolver.

8. Matriz de verificación de la evidencia

No.	Indicador de logro	Evidencia dentro del documento	Estado
1	Describe los componentes a realizar en el diseño front-end, según las características del proyecto a desarrollar.	Secciones 1, 3 y 5: módulos de SAAAF, arquitectura y descripción detallada de componentes estructurales, reutilizables y de dominio.	Cumple
2	Demuestra coherencia en la selección de los componentes, cumpliendo con las necesidades del software.	Secciones 2 y 4: procedimiento de definición y matriz que relaciona cada necesidad funcional con componentes y justificación.	Cumple

Conclusiones

- La definición de componentes para SAAAF parte de necesidades funcionales concretas y no solamente de la apariencia de las pantallas.
- Separar componentes estructurales, reutilizables y de dominio permite reducir duplicación y facilita mantener el front-end a medida que el sistema crece.
- Componentes como DataTable, FormField, FilterBar, ModalDialog y ToastNotification pueden reutilizarse en varios módulos, mientras que AssetForm, AssignmentForm y ReturnForm encapsulan reglas propias del negocio.
- La matriz de trazabilidad evidencia coherencia entre las funciones del sistema y los componentes propuestos, cubriendo los dos indicadores definidos en el instrumento de evaluación.
- La organización por componentes es compatible con el front-end React/Next.js trabajado para SAAAF y facilita pruebas, validaciones y evolución futura del proyecto.

Referencias

React. Documentación oficial: Describing the UI. <https://react.dev/learn/describing-the-ui>

React. Documentación oficial: Sharing State Between Components. <https://react.dev/learn/sharing-state-between-components>

React. Documentación oficial: Responding to Events. <https://react.dev/learn/responding-to-events>

Next.js. Documentación oficial. <https://nextjs.org/docs>

Servicio Nacional de Aprendizaje - SENA. Lineamientos de la evidencia GA7-220501096-AA4-EV02.

Resultado de la definición

La propuesta establece una base de componentes coherente con los módulos del sistema SAAAF y deja documentada la justificación de cada selección para orientar la implementación del front-end.